

Lab 20: Operator Overloading

- Due Friday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 15 at 6pm - Jul 17 at 11:59pm

Learning Objectives

- Practice operator overloading.
- Explore contains relations in Object Oriented Programming

Completion Requirements

- Submit Lab20.py

Q1: Vectors

1. Write a class **Vector** (in Lab20.py) with the following requirements.

- **Please name your list variable in the Vector class to be “elements” (it should be `self.elements = ...`)**

1. Stores the vector elements in the list type data attribute.
2. The constructor should accept a list argument that is stored into the internal list attribute.
3. Implement `__add__`, `__sub__`, `__mul__`, `__truediv__`, `__eq__`, `__str__`. These methods should override the `+`, `-`, `*`, `/`, `==` operators. `__str__` adds support for str method and print. The mathematical operations will perform element by element operations. Division by zero will result in **None**.

Note: For `+`, `-`, `*` and `/`, you need to create a new Vector and return the new vector. Do not modify the given vectors.



The example shows the usage.

Example 1:

```
>> v1 = Vector([2, 3, 4])
>> v2 = Vector([1, 2, 3])
>> v3 = v1 + v2
>> print(v3)
[3, 5, 7]
```

```
>> v4 = v1 - v2
>> print(v4)
[1, 1, 1]
```

Q2: Employee Directory

Write a class named `Employee` with the following attributes:

1. Name
2. ID number
3. Department
4. Job title

Implement `__str__` function that prints the `Employee` information:

Example 1:

```
>> e = Employee('Tom', 123, 'Accounting', 'Payroll Manager')
>> print(e)
Name          : Tom
ID            : 123
Department    : Accounting
Job Title     : Payroll Manager
```

Now implement another class **`Employee_Directory`** with the following attributes (Not the name of the attributes can be different as explained in the last lab):

1. Company Name
2. employees

The attribute `employees` should be a dictionary indexed by `Employee` id. The following methods must be supported.

1. `__str__()`
2. `add_employee(e)` where `e` is of `Employee` type

Example 2:

```
>> e1 = Employee('Tom', 123, 'Tom & Jerry', 'Cat')
>> e2 = Employee('Jerry', 321, 'Tom & Jerry', 'Mouse')
>> c = Employee_Directory("MGM cartoons")
>> c.add_employee(e1)
>> c.add_employee(e2)
>> print(c)
Company : MGM cartoons
```

```
ID : 123
Name      : Tom
ID        : 123
Department : Tom & Jerry
Job Title  : Cat
```

```
ID : 321
Name      : Jerry
ID        : 321
Department : Tom & Jerry
Job Title  : Mouse
```

This tool needs to be loaded in a new browser window

Load Lab 20: Operator Overloading in a new window

